

STOQUIFY

Stoquify Internal Activation Gate Unblocking Analysis And Technical Execution Plan

EVIDENCE-BASED REVIEW REPORT

Technical gate analysis, control-plane design, implementation sequence, certification matrix, and go/no-go decision.

CURRENT DECISION: BLOCKED FOR INTERNAL ACTIVATION

Prepared 23 July 2026

Stoquify Agent Runtime Phase 2A

Controlled document. No activation or production mutation is authorized by this artifact.

Date: 2026-07-23

Review scope: Stoquify Agent Runtime Phase 2A Command Agent

Selected skills:

1. 016-aqstoqflow-ai-copilot-guardrails
2. 017-aqstoqflow-enterprise-release-gate

Final decision: BLOCKED

Engineering baseline: **PASSED**

Controlled internal activation: **NOT APPROVED**

1. Executive Explanation

The Phase 2A Command Agent is technically credible as a deterministic, provider-free, read-only feature. Its current runtime checks trusted tenant context, RBAC, module entitlement, pilot organization and role allowlists, active definitions, an exact code-to-database manifest match, tool policy, evidence citations, redaction, bounded execution time, and idempotent replay. The fresh review also passed the Phase 2A static gate, tool registry gate, prohibited-action gate, and all 46 focused tests.

That engineering result does not authorize internal use.

Internal activation remains blocked because Stoquify cannot yet prove who approved the release, who owns rollout and rollback, whether the deployed definitions still match the approved release, whether a production scheduler is running, whether alerts reach an accountable operator, or whether the real enabled pilot works through the browser. These are not administrative niceties. They are the controls that turn functioning code into an accountable, observable, reversible service.

The smallest professional path is not to add another agent framework. It is to add a narrow agent release-control plane around the runtime already built:

1. Persist an immutable activation package for one code manifest and one pilot scope.
2. Record separate product and security approvals against that package.
3. Record named primary and backup owners who accept their duties.
4. Route every state change through one guarded transition service.
5. Extend the existing Workflow Assurance incident spine to reconcile the agent

control plane and deliver operational alerts.

6. Certify the exact approved manifest in a real enabled-pilot Playwright environment.
7. Permit ACTIVE_INTERNAL only when all of those records are current and healthy.

No activation was performed during this review.

2. Evidence Reviewed

Runtime and control evidence

- services/agents/agent-rollout.service.ts:15-53
- services/agents/agent-definition.service.ts:95-141
- services/agents/agent-runner.service.ts:104-200
- services/agents/agent-runner.service.ts:209-385
- services/agents/agent-run-governance.service.ts:80-108

- `services/agents/agent-reconciler-auth.service.ts:5-38`
- `services/agents/agent-execution-control.service.ts:17-52`
- `services/agents/agent-metrics.service.ts:5-33`
- `services/agents/agent-policy.service.ts:29-90`
- `actions/agents/command-agent.actions.ts:22-114`
- `app/api/internal/agents/reconcile-abandoned/route.ts:13-55`
- `scripts/agent-runtime-phase-2a-provision.js:6-103`
- `tests/e2e/command-agent-kill-switch.spec.ts:5-30`
- `playwright.config.ts:29-83`
- `prisma/schema.prisma:6889-7180`

Existing platform infrastructure considered for reuse

- `services/assurance/assurance-incident.service.ts:662-716`
- `services/assurance/assurance-incident-contracts.ts:29-60`
- `prisma/schema.prisma:6563-6770`
- `lib/logger.ts:12-33`
- `.github/workflows/ci.yml:18-112`

Prior Phase 2A evidence

- `docs/agents-runtime/STOQUIFY_AGENT_RUNTIME_PHASE_2A_COMMAND_AGENT_DESIGN_FREEZE_2026-07-22.md`
- `what-next/agents-runtime/STOQUIFY_AGENT_RUNTIME_PHASE_2A_COMMAND_AGENT_EXECUTION_REPO_RT_2026-07-22.md`
- `what-next/agents-runtime/STOQUIFY_AGENT_RUNTIME_PHASE_2A_ENTERPRISE_RELEASE_GATE_2026-07-22.md`
- `what-next/agents-runtime/STOQUIFY_AGENT_RUNTIME_PHASE_2A_GUARDRAIL_AND_ENTERPRISE_GATE_RERUN_2026-07-22.md`

Fresh verification

Verification	Result
Phase 2A static gate	Passed
Agent tool-registry gate	Passed
Prohibited-action gate	Passed
Focused agent and Daily Digest Jest suites	17/17 suites, 46/46 tests passed
Provision command without flags	Dry-run, activate: false, SHADOW
Inspected PostgreSQL agent definition	DRAFT, SHADOW, READ_ONLY
Inspected PostgreSQL skill definition	DRAFT, version 1
Inspected PostgreSQL tool definition	DRAFT, READ_ONLY
Inspected environment rollout	Unset
Inspected pilot organization and role counts	Zero
Inspected reconciler secret	Not configured
Inspected production alert transport	Not configured

The PostgreSQL and environment checks apply to the inspected workspace environment. No evidence of a production deployment, production secret manager, production scheduler, or production alert route exists in the repository.

The current knowledge graph contains the Daily Digest component but does not yet represent the new services/agents Phase 2A slice. Runtime source files were therefore treated as the primary architecture evidence.

3. Current-State Interpretation

3.1 What DRAFT means

AgentDefinition, AgentSkillDefinition, and AgentToolDefinition use AgentDefinitionStatus. The inspected records exist, but all three are DRAFT.

The runtime requires all three records to be ACTIVE before execution: resolveActiveCommandAgentDefinition rejects a missing or non-active agent, skill, or tool with DEFINITION_NOT_ACTIVE. It then compares rollout mode, risk, tool keys, skill keys, prompt hash, permission, service owner, module, tool type, and schema hashes against the code manifest.

Therefore, DRAFT means:

- the governed metadata is present for review and testing;
- it is not authorized for runtime execution;
- editing it does not constitute activation;
- its presence in PostgreSQL is not proof of approval or deployment readiness.

3.2 What dry-run provisioning means

The provision script has two independent flags:

- --apply permits database upserts;
- --activate changes the target status from DRAFT to ACTIVE.

Without --apply, the command only prints the intended manifest and exits. Without --activate, an applied provision writes DRAFT.

There is an important audit clarification: the inspected local PostgreSQL database already contains DRAFT/SHADOW agent, skill, and tool records. Thus, a DRAFT provision was applied locally at some point, while the activation command

remains a dry-run and `activate: false`. It would be inaccurate to say that no provisioning record exists. The accurate statement is:

The local governance definitions are provisioned as inactive DRAFT records; the activation operation has not been applied, and no production deployment evidence has been provided.

3.3 What `activate: false` means

`activate` is currently a command-line boolean in the provisioning script, not a database field. The script translates it as follows:

```
activate false -> definition status DRAFT
activate true  -> definition status ACTIVE
```

The current design therefore has no persisted approval object behind `activate: true`. A person with production database credentials could run `--apply --activate` and change all three records directly. Runtime manifest checks would still protect against code drift, but they would not prove product approval, security approval, named ownership, browser certification, or release authorization.

This activation path must be replaced before internal activation.

3.4 What blocked internal activation means

The feature may remain visible in a disabled state, and engineers may continue unit, integration, migration, shadow, and non-production certification work. It must not render an enabled internal brief to users.

The current fail-closed layers are:

1. Missing or invalid rollout environment value resolves to off.
2. Empty organization and role allowlists deny all users.
3. The kill switch takes precedence.
4. DRAFT definitions fail the active-definition gate.
5. A manifest mismatch fails closed.
6. RBAC and module-entitlement checks run at protected boundaries.

These layers appropriately keep the incomplete release package from becoming a user exposure.

3.5 Controlled pilot versus production readiness

An internal pilot is a narrowly authorized service for one named organization and specified roles, during a bounded window, with named owners, tested rollback, live monitoring, and evidence retention. It may still use a deterministic provider-free agent and conservative service-level objectives.

Production or general availability additionally requires multi-tenant rollout management, capacity and cost controls, broader role certification, support training, longer reliability evidence, formal incident response, data-retention validation, provider governance if a model is introduced, and legal or regulatory validation where applicable.

Passing the internal-pilot gate does not approve Phase 2B, external model use, write tools, ledger posting, statutory filing, payroll decisions, or permission changes.

4. Gate Summary

Gate	Current status	Principal reason
Product and security approval	Blocked	No persisted or signed approval package exists
Named operational owners	Blocked	No people, backups, acknowledgements, or escalation routes are recorded
Reconciliation schedule and secret	Blocked	Endpoint exists, but only reconciles abandoned runs; no deployed scheduler or secret exists
Production alert transport and ownership	Blocked	Metrics are durable, but logger sink is no-op and existing alert deliveries have no external dispatcher
Enabled-pilot Playwright certification	Blocked	Only kill-switch browser coverage exists; the configured test server forces the kill switch on

5. Gate 1: Recorded Product And Security Approval

Plain-language meaning

Stoquify needs a durable record showing exactly what product and security approved. An email or verbal instruction is insufficient because the code, manifest, tenant scope, roles, residual risks, and activation window can change independently.

Purpose and risk controlled

This gate prevents:

- activation of a different commit from the one reviewed;
- silent expansion to another tenant or role;
- use of an unapproved tool, skill, model, or policy;
- acceptance of residual risks without an accountable approver;
- activation after an approval expires or is revoked;
- direct database activation with no release evidence.

Current gap

The agent schema stores runtime definitions and execution evidence but has no release package, approval, owner, certification, or transition record. The provision script sets statuses directly and does not consult an approval service.

Required implementation

Add a small release-control schema:

```
enum AgentActivationState {
    DRAFT
    REVIEWED
    APPROVED
    PROVISIONED_INACTIVE
    PILOT_CERTIFIED
    ACTIVE_INTERNAL
    SUSPENDED
    RETIRED
}

enum AgentApprovalType {
    PRODUCT
    SECURITY
}

enum AgentApprovalDecision {
    APPROVED
    REJECTED
    REVOKED
}

enum AgentOwnerResponsibility {
    ROLLOUT
    ROLLBACK
    SUPPORT
    PILOT
    SECURITY_INCIDENT
    ON_CALL_BACKUP
}

model AgentActivationPackage {
    id                String @id @default(cuid())
    agentKey          String
    releaseVersion    String
    environment       String
    commitSha         String
    manifestHash      String
    manifest          Json
    pilotOrganizationId String
    allowedRoleCodes  String[]
    activationStartsAt DateTime
}
```

```

    activationEndsAt    DateTime
    residualRisks       Json?
    state               AgentActivationState @default(DRAFT)
    version             Int @default(1)
    requestedById       String
    activatedById       String?
    activatedAt         DateTime?
    suspendedAt        DateTime?
    suspensionReason    String?
    createdAt           DateTime @default(now())
    updatedAt           DateTime @updatedAt
  }

  model AgentActivationApproval {
    id                String @id @default(cuid())
    packageId         String
    approvalType      AgentApprovalType
    decision           AgentApprovalDecision
    approverId        String
    evidenceHash       String
    riskAcceptance    Json?
    decidedAt         DateTime
    expiresAt          DateTime
    revokedAt          DateTime?
    createdAt          DateTime @default(now())
  }

  model AgentActivationOwner {
    id                String @id @default(cuid())
    packageId         String
    responsibility     AgentOwnerResponsibility
    primaryUserId      String
    backupUserId       String?
    escalationRouteRef String
    acceptedAt         DateTime?
    validUntil         DateTime?
    createdAt          DateTime @default(now())
    updatedAt          DateTime @updatedAt
  }

  model AgentPilotCertification {
    id                String @id @default(cuid())

```

```

    packageId         String
    commitSha         String
    manifestHash      String
    suiteVersion      String
    ciRunId           String
    result            String
    reportHash        String
    passedAt          DateTime
    expiresAt          DateTime
    createdAt          DateTime @default(now())
  }
}

```

The final Prisma design must add relations, uniqueness, indexes, and deletion rules. The important invariants are:

- approvals are append-only;
- product and security approvals are separate;
- an approver cannot approve a different manifest hash;
- rejected, revoked, or expired approvals cannot be used;
- the pilot organization, role list, code commit, and manifest become immutable after

REVIEWED;

- only one package may be ACTIVE_INTERNAL for an agent and environment;

- all state changes write an `AuditLog` record;
- sensitive contact details and webhook URLs are represented by secret or directory

references, not copied into release metadata.

API and permission boundary

Add human-operated control-plane permissions:

- `agent.release.view`
- `agent.release.prepare`
- `agent.release.approve.product`
- `agent.release.approve.security`
- `agent.release.activate`
- `agent.release.suspend`

Expose protected control-plane operations, not raw Prisma access:

```
POST /api/internal/agents/releases
POST /api/internal/agents/releases/{id}/review
POST /api/internal/agents/releases/{id}/approve
POST /api/internal/agents/releases/{id}/provision-shadow
POST /api/internal/agents/releases/{id}/certify
POST /api/internal/agents/releases/{id}/activate
POST /api/internal/agents/releases/{id}/suspend
```

The agent runtime must never receive these permissions. Approval and activation must require fresh authentication, an idempotency key, optimistic version checking, and separation of duties.

Activation transaction

Replace direct status writes in the provisioning script with one canonical service:

```
await db.$transaction(async (tx) => {
  const release = await lockActivationPackage(tx, releaseId)
  assertExpectedVersion(release, expectedVersion)
  assertState(release, "PILOT_CERTIFIED")
  assertActivationWindowOpen(release)
  assertManifestHash(release, CODE_MANIFEST_HASH)
  await assertValidApprovals(tx, release, ["PRODUCT", "SECURITY"])
  await assertAcceptedOwners(tx, release, REQUIRED_OWNER_RESPONSIBILITIES)
  await assertFreshPilotCertification(tx, release)
  await assertFreshReconcilerHeartbeat(tx, release)
  await assertAlertTransportHealthy(tx, release)

  await activateExactDefinitions(tx, release.manifest)
  await tx.agentActivationPackage.update({
    where: { id: release.id, version: expectedVersion },
    data: {
      state: "ACTIVE_INTERNAL",
      activatedById: actorId,
      activatedAt: now,
      version: { increment: 1 },
    },
  },
  {})
  await writeActivationAudit(tx, release, actorId, idempotencyKey)
})
```

The old `--activate` switch should be removed or changed so it can only call this service with a release ID. It must no longer set definition statuses itself.

Completion evidence

- Applied migration and schema validation.
- Product approval record tied to the exact manifest hash.
- Security approval record tied to the exact manifest hash.
- Approval evidence hashes referencing the signed design freeze and threat review.
- Negative tests for missing, rejected, revoked, expired, same-person, and wrong-hash

approvals.

- Audit evidence for prepare, review, approve, reject, revoke, activate, suspend, and retire transitions.

- A static gate proving no script directly sets agent definitions to ACTIVE.

Acceptance criteria

Gate 1 passes only when a specific package has two valid approvals, immutable scope, unexpired evidence, and no path can activate definitions without the guarded service.

Accountable roles

- Product approver: accountable for user value, pilot scope, and residual product risk.
- Security approver: accountable for threat controls and residual security risk.
- Release-control owner: responsible for service implementation and evidence.

Rollback condition

Any approval revocation, expiry, manifest mismatch, or unauthorized transition must block activation. If already active, it must move the package to SUSPENDED, pause definitions, preserve all run records, and emit a critical incident.

6. Gate 2: Named Rollout, Rollback, Support, Pilot, And Incident Owners

Plain-language meaning

Every operational duty must be accepted by a named person and backup. A team name such as "Engineering" does not answer who has authority to activate, who presses the kill switch, or who responds when a pilot user reports unsafe output.

Purpose and risk controlled

This gate prevents:

- an alert with no accountable responder;
- delayed rollback while teams debate authority;
- unsupported pilot users;
- incidents remaining open across handoffs;
- activation during a window when the required people are unavailable.

Current gap

The reviewed documents describe owner roles but record no people, backups, acceptance timestamps, escalation route, on-call schedule, or activation window.

Required RACI

The following cells must contain actual directory user IDs before activation. TBD remains a blocking value.

Activity	Responsible	Accountable	Consulted	Informed
Approve pilot value and scope	Product approver	Product owner	Pilot owner, support	Finance and operations leadership
Approve security boundary	Security approver	Security owner	Platform, privacy, audit	Product owner
Execute activation	Rollout owner	Release owner	Security, pilot owner	Support, on-call
Decide and execute rollback	Rollback owner	Release owner	Security incident owner	Product, pilot, support
Support pilot users	Support owner	Pilot owner	Rollout owner	Product owner
Validate pilot acceptance	Pilot owner	Product owner	Named pilot users	Release owner
Triage security incidents	Security incident owner	Security owner	Rollback owner, SRE	Product owner
Respond outside primary coverage	On-call backup	Operations owner	Security incident owner	Release owner

Required owner record

For every responsibility, persist:

- primary directory user ID;
- backup directory user ID;
- accepted-at timestamp;
- valid-until timestamp when temporary;
- escalation route reference;
- duty description version;
- timezone and coverage window;
- acknowledgement and response objectives.

Do not store personal phone numbers or webhook secrets in the release table. Store a reference to the approved directory, on-call, or secret-manager record.

Runbooks that owners must acknowledge

- controlled activation;
- kill switch and package suspension;
- definition rollback;
- reconciler failure;
- alert-delivery failure;
- tenant-isolation incident;
- unsafe-output or prohibited-tool attempt;
- provider outage, reserved for a future model-backed phase.

Completion evidence

- Six accepted owner assignments with primary and backup people.
- A RACI artifact signed or approved through the release package.
- On-call route test acknowledged by the named recipient.
- A timed rollback rehearsal with evidence.

- Support handoff containing known limitations, pilot scope, and escalation path.

Acceptance criteria

Gate 2 passes only when every required responsibility has an accepted primary owner, a reachable backup, a valid escalation route, and coverage for the entire activation window.

Accountable role

The release owner is accountable for completeness. Each named person is responsible for accepting and performing the assigned duty.

Rollback condition

If an owner withdraws, becomes unreachable, or loses authority during the pilot, new runs must be suspended until a replacement accepts the duty.

7. Gate 3: Deployed Reconciliation Schedule And Secret

Plain-language meaning

Stoquify needs an independent, recurring control that proves the deployed runtime still matches the approved release. An endpoint that can be called is not a scheduler, and a scheduler with no monitored secret is not an operational control.

Current implementation

The existing endpoint:

- fails with 503 when `STOQUIFY_AGENT_RECONCILER_SECRET` is absent;
- requires a minimum 32-character bearer secret;
- compares equal-length secrets using `timingSafeEqual`;
- bounds the stale-run threshold and batch size;
- marks old `RUNNING` runs as safely failed.

This is useful but incomplete. It reconciles abandoned execution rows only. It does not compare the approved manifest to definitions, pilot scope, approvals, owners, certification, rollout mode, permissions, policies, or activation state.

No scheduler, secret, cadence monitor, infrastructure declaration, or production heartbeat was found.

Required reconciliation architecture

Use one scheduled agent-runtime reconciliation entry point containing two checks:

1. Execution reconciliation: recover abandoned `AgentRun` records.
2. Control-plane reconciliation: detect release, definition, scope, policy, approval,

certification, and operational drift.

Prefer platform workload identity or signed OIDC service authentication. If the existing bearer design is retained for the first pilot:

- generate at least 32 random bytes;
- store it only in the deployment secret manager;
- inject it into the web service and scheduler;
- never place it in source, CI logs, reports, or screenshots;
- rotate it before pilot and on any suspected exposure;

- restrict the route at the network layer where supported;
- rate-limit failed authentication attempts;
- log only a safe authorization failure code.

Schedule

Recommended initial pilot values:

- scheduler cadence: every 5 minutes;
- abandoned-run threshold: 15 minutes;
- missing-heartbeat alert: no successful run for 10 minutes;
- reconciliation batch limit: 100, bounded to 500;
- request timeout: 30 seconds;
- retries: three attempts with exponential backoff and jitter;
- idempotency key: `agent-reconcile:{environment}:{scheduled-window}`.

The actual provider must be recorded. Examples include a Kubernetes CronJob, Azure Container Apps Job, managed cloud scheduler, or hosting-platform cron. A GitHub Action is not the production scheduler.

Reconciliation checks

For each approved or active package, compare:

- package commit SHA and manifest hash to the deployed build;
- package manifest to the code-backed manifest;
- expected agent, skill, and tool records to PostgreSQL;
- definition status and rollout mode;
- agent skill version and prompt hash;
- allowed tool and skill keys;
- tool service owner, module, permission, type, risk, and schema hashes;
- pilot organization and roles to the approved package;
- active environment rollout to the package state;
- product and security approvals for expiry, revocation, and exact hash;
- required owners for acknowledgement and validity;
- Playwright certification for commit, manifest hash, result, and expiry;
- duplicate or unauthorized active packages;
- stale, missing, or extra active definitions;
- open critical policy incidents;
- abandoned executions and exhausted retries;
- alert transport heartbeat and pending/failed delivery backlog.

Fail-closed policy

The reconciler must not silently rewrite drift into the expected state.

- Informational drift: record and notify.

- Warning drift: block the next promotion.
- High drift: deny new runs for the affected package.
- Critical drift, including unauthorized activation, cross-tenant scope expansion, or

invalid approval: atomically move the package to SUSPENDED, pause exact definitions, preserve evidence, and emit a critical incident.

Reconciliation pseudocode

```
export async function reconcileAgentRuntime(input: ScheduledJobInput) {
  const job = await beginIdempotentAssuranceRun(input.idempotencyKey)

  try {
    const build = loadDeployedBuildIdentity()
    const packages = await loadGovernedPackages()
    const findings = []

    for (const release of packages) {
      const actual = await loadAgentControlPlane(release.agentKey)
      findings.push(...compareManifest(release, build, actual))
      findings.push(...validateApprovals(release))
      findings.push(...validateOwners(release))
      findings.push(...validatePilotScope(release))
      findings.push(...validateCertification(release))
      findings.push(...validateAlertHealth(release))
    }

    const recovered = await reconcileAbandonedAgentRuns({
      olderThan: input.abandonedBefore,
      limit: input.limit,
    })

    await persistAssuranceFindings(job, findings, { recovered })
    await suspendCriticalDrift(findings)
    await enqueueFindingAlerts(findings)
    await completeAssuranceRun(job, "COMPLETED")
  } catch (error) {
    await failAssuranceRun(job, safeFailureCode(error))
    await enqueueReconcilerFailureAlert(job)
    throw error
  }
}
```

Reuse of existing Stoquify infrastructure

Extend `WorkflowAssuranceWorkflow` with `AGENT_RUNTIME`, add versioned agent-runtime check definitions, and reuse:

- `WorkflowAssuranceCheckRun` for heartbeat and result evidence;
- `WorkflowAssuranceIncident` for assignment and lifecycle;
- `WorkflowAssuranceIncidentEvent` for history;
- `WorkflowAssuranceAlertDelivery` for deduplicated delivery;
- `WorkflowAssuranceWaiver` only for non-critical, explicitly waivable findings.

Tenant isolation, prohibited writes, missing audit evidence, invalid approvals, and unauthorized activation must never be waivable.

Completion evidence

- Scheduler infrastructure declaration reviewed and deployed.
- Secret-manager reference and rotation record, without the secret value.

- Successful production-like scheduler invocation.
- Two consecutive successful heartbeats.
- Injected manifest drift detected and recorded.
- Injected critical drift automatically suspended in a non-production rehearsal.
- Failed schedule and bad-secret alerts delivered to the named owner.

Acceptance criteria

Gate 3 passes when the job is deployed, authenticated, monitored, idempotent, and able to detect both abandoned runs and control-plane drift within the approved detection window.

Accountable role

Operations or SRE owner.

Rollback condition

If the job causes unexpected load or false suspensions, disable its repair/suspension action but keep detection and alerting active. If heartbeats stop, the runtime must fail closed according to the release policy.

8. Gate 4: Production Alert Transport And Ownership

Plain-language meaning

Stoquify already records useful run and incident data. It does not yet prove that a human receives, acknowledges, escalates, and resolves an operational problem.

Current implementation

- AgentRun durably records status, failure code, duration, tool count, evidence count, redaction count, and stale state.
- policy denials create AgentPolicyIncident.
- agent metrics call the generic logger.
- lib/logger.ts defaults to a no-op sink.
- Workflow Assurance can create an IN_APP delivery in PENDING.
- no external delivery worker was found for pending Workflow Assurance deliveries.
- no agent dashboard, alert thresholds, recipient, acknowledgement objective, or escalation route is recorded.

Required alert path

```
Agent runtime / reconciler
-> durable run or assurance finding
-> Workflow Assurance incident
-> deduplicated alert delivery
-> external transport and in-app control center
-> named owner acknowledgement
-> escalation if overdue
-> resolution evidence
-> post-incident review when required
```

For the first pilot, configure:

1. An authoritative external on-call transport such as a managed incident service or approved operations webhook.
2. The existing in-app assurance queue as the durable operational record.
3. Structured JSON logs to the deployed platform log collector.

Do not make the logger the sole incident record. Logs may be sampled or unavailable; the PostgreSQL incident and delivery records are the durable control.

Alert delivery worker

Implement one platform-wide Workflow Assurance delivery worker rather than an agent-specific webhook client. It should:

- claim pending deliveries with database-safe concurrency;
- use a bounded timeout;
- send only safe, redacted fields;
- update DELIVERED, FAILED, SKIPPED, or SUPPRESSED;
- retry transient failures with exponential backoff;
- dead-letter exhausted deliveries;
- deduplicate by incident, channel, and dedupe key;
- alert through a secondary route when the primary transport fails;
- record external incident ID, delivery timestamp, and safe failure code;
- never persist or log webhook secrets.

Initial severity and threshold policy

Condition	Severity	Initial pilot action
Unauthorized activation attempt	Critical	Suspend, page security and rollback owners
Active manifest or tenant-scope drift	Critical	Suspend, page security and rollout owners
Approval revoked or expired while active	Critical	Suspend, page release owner
Audit write failure on activation or run	Critical	Block operation, page security owner
Missing reconciliation heartbeat for 10 minutes	High	Deny promotion; suspend if policy requires
Any cross-tenant authorization anomaly	Critical	Suspend and initiate security incident
Three policy denials by one actor in 5 minutes	High	Alert security and pilot owner
More than three failed runs in 15 minutes	High	Alert support and rollout owner
Timeout or failure rate above 5%, minimum 20 runs	High	Alert support; consider suspension
Any unsafe feedback classification	High	Alert security and product owners
Stale output above 20%, minimum 10 runs	Warning	Alert pilot and data owners
Pending or failed alert backlog above zero for 10 minutes	High	Alert through secondary channel

Thresholds should be reviewed after the first pilot week. Security invariants remain event-based and must not wait for a volume threshold.

Minimum dashboards and service objectives

Dashboard panels:

- runs by status and agent;
- p50, p95, and maximum duration;
- timeout and failure rate;
- policy denials by safe code;
- stale and partial output rate;
- evidence and redaction counts;
- feedback classifications;
- abandoned run count;
- manifest drift findings;
- last successful reconciliation;
- open incidents by severity and owner;
- pending and failed alert deliveries.

Pilot objectives:

- unauthorized tenant or role access denied: 100%;
- prohibited write execution: 0;
- activation and run audit persistence: 100%;
- critical drift detection: within 5 minutes;
- alert acknowledgement: Critical within 15 minutes, High within 30 minutes;
- successful reconciliation heartbeat: at least every 10 minutes;
- Command Agent p95 execution: below 5 seconds;
- execution success rate: at least 98%, excluding expected policy denials.

Completion evidence

- Named primary and secondary alert routes.
- Successful test alerts through both routes.
- Owner acknowledgement recorded.
- Dashboard links and query definitions.
- Alert redaction test proving no raw prompt, output, token, bank, payroll, or personal

payload leaves Stoquify.

- Delivery retry and dead-letter test.
- Alert transport outage rehearsal.

Acceptance criteria

Gate 4 passes only when a production-like event reaches the named owner, is acknowledged, escalates correctly when ignored, and retains complete resolution evidence.

Accountable role

Operations owner, with security ownership for security-class incidents.

Rollback condition

If the primary transport fails, the durable in-app incident remains open and the secondary transport is used. If no alert route is healthy, internal activation must remain blocked or be suspended.

9. Gate 5: Enabled-Pilot Playwright Certification

Plain-language meaning

Unit tests prove functions and components. The enabled-pilot certification must prove that the assembled application works with real authentication, tenant data, active governance records, pilot scope, protected actions, PostgreSQL evidence, and browser behavior.

Current implementation

The current Playwright suite proves that the authenticated Daily Digest:

- renders the Command Agent surface;
- disables Generate Brief when the kill switch is on;
- explains the disabled state;
- shows the prohibited-authority warning;
- has no horizontal overflow on desktop and mobile.

The Playwright web server explicitly sets `STOQUIFY_COMMAND_AGENT_KILL_SWITCH: "1"`. There is no enabled-pilot project. The CI workflow runs repository verification and the Close Assurance browser smoke, but no Command Agent Playwright job.

Test environment design

Create a dedicated ephemeral PostgreSQL database and seed:

- pilot organization A;
- non-pilot organization B;
- approved owner/manager pilot user;
- same-organization denied-role user;
- non-pilot user with otherwise similar permissions;
- real Daily Digest evidence for fresh, stale, partial, empty, and blocked cases;
- one approved activation package tied to the CI commit and code manifest;
- accepted E2E owner assignments;
- E2E-only product and security approval identities;
- active definitions provisioned through the same guarded transition service;
- cleanup limited to the unique E2E fixture namespace.

Authentication, tenant resolution, RBAC, module access, definitions, run persistence, evidence links, and audit records must be real. Controlled fault injection may be used for timeout and transport failures, but it must be available only in a non-production test environment and must not mock the authorization boundary.

Playwright projects

Add separate projects:

- `command-agent-pilot-enabled-desktop`;

- `command-agent-pilot-enabled-mobile`;
- `command-agent-pilot-denied-tenant`;
- `command-agent-pilot-denied-role`;
- `command-agent-pilot-rollback`;
- `retain command-agent-kill-switch-desktop`;
- `retain command-agent-kill-switch-mobile`.

The enabled server must run with kill switch off and an exact pilot scope. The kill-switch projects must use a separately started server or job because process environment values cannot be safely changed inside a running test server.

Certification matrix

Scenario	Required evidence
Approved pilot user	Panel enabled; brief generated; run and evidence rows persisted
Non-pilot tenant	Generate denied; no cross-tenant data or run receipt returned
Denied role	Generate denied despite authenticated access
Missing permission	Protected action returns safe forbidden response
Module unavailable	Surface or action fails closed
DRAFT definition	Execution denied with safe governance message
Manifest mismatch	Execution denied and reconciliation incident created
Read-only tool	Brief succeeds; no business tables mutate
Prohibited tool request	Static and runtime policy block; high incident recorded
Ledger, filing, permission, and Prisma write attempts	Structurally unavailable and rejected
Approval-required action bypass	No route or generic executor is reachable
Evidence navigation	Link remains tenant-protected and resolves to permitted source
Stale or partial evidence	Visible limitation and stale/partial state
Empty result	Accessible empty state, no fabricated priority
Timeout	Safe failure state, retry available, AGENT_TIMEOUT persisted
Replayed request	Same scoped receipt, no duplicate run
Feedback	Bounded classification persists for actor and run
Audit evidence	Protected-action audit plus run provenance present
Package suspension	New run denied immediately
Kill switch	Surface visibly disabled on desktop and mobile
Rollback	Package and definitions suspended; prior evidence retained
Keyboard operation	Generate, evidence, feedback, and retry are keyboard reachable
Accessibility	No critical automated violation; labels and focus state present
Layout	No incoherent overlap or horizontal overflow at certified viewports

CI and evidence

Add a required `command-agent-pilot-certification` CI job after repository verification:

1. create isolated database;
2. deploy migrations;

3. seed pilot and denied fixtures;
4. create the release package for the current commit;
5. provision through the guarded shadow and certification path;
6. start the enabled server;
7. run the enabled and denied projects;
8. start or reuse a kill-switch server;
9. run rollback projects;
10. export JUnit, HTML, trace, screenshot, video, and a machine-readable certification

summary;

11. hash the report and record AgentPilotCertification;
12. clean the isolated fixtures.

Retain failure traces, screenshots, and videos for at least 30 days for the pilot. Never include secrets or sensitive source payloads in artifacts.

Completion evidence

- Required CI job passes for the exact commit and manifest hash.
- Certification record is current and unexpired.
- Desktop and mobile enabled path passes.
- Tenant and role negative tests pass.
- Suspension and kill-switch tests pass.
- Audit and database assertions pass.
- No business-table mutations are detected.

Acceptance criteria

Gate 5 passes only when the exact package intended for activation has a valid, machine-readable certification produced by the real enabled-pilot environment.

Accountable role

QA or release-verification owner, with product acceptance by the pilot owner.

Rollback condition

Any failed required scenario invalidates certification. The package remains or returns to PROVISIONED_INACTIVE or SUSPENDED; definitions must not be enabled for internal rendering.

10. Activation State Machine

Use the state machine on AgentActivationPackage. Keep the simpler definition status as an execution projection controlled only by the transition service.

```

DRAFT
-> REVIEWED
-> APPROVED
-> PROVISIONED_INACTIVE
-> PILOT_CERTIFIED
-> ACTIVE_INTERNAL
-> SUSPENDED
-> RETIRED

```

Transition contract

Transition	Authorized actor	Preconditions	Definition projection
DRAFT -> REVIEWED	Release preparer	Manifest, commit, scope, window, risks complete	DRAFT
REVIEWED -> APPROVED	System after approvers	Valid product and security approvals; owners assigned	DRAFT
APPROVED -> PROVISIONED_INACTIVE	Rollout owner	Exact manifest; scheduler and alert preflight; non-production or shadow scope	ACTIVE + SHADOW, no rendered output
PROVISIONED_INACTIVE -> PILOT_CERTIFIED	CI certification identity	Required Playwright suite passes for same commit and hash	ACTIVE + SHADOW
PILOT_CERTIFIED -> ACTIVE_INTERNAL	Rollout owner with fresh auth	Window open; approvals, owners, heartbeat, alert route, and cert valid	ACTIVE + INTERNAL
ACTIVE_INTERNAL -> SUSPENDED	Rollback or security owner; automatic reconciler	Manual decision or critical invariant failure	PAUSED and no new runs
SUSPENDED -> ACTIVE_INTERNAL	Rollout owner	Root cause closed; unchanged package or new approvals and certification	ACTIVE + INTERNAL
Any non-retired -> RETIRED	Product and release owners	Pilot closed; retention verified	RETIRED

Automatic suspension conditions

- manifest or commit mismatch;
- unauthorized pilot organization or role expansion;
- product or security approval expiry or revocation;
- missing required owner during the active window;
- invalid or expired Playwright certification;
- critical tenant, audit, prohibited-write, or activation incident;
- missing reconciliation heartbeat beyond the approved tolerance;
- no healthy alert route;
- explicit kill-switch or rollback action.

Emergency behavior

The environment kill switch remains the highest-priority coarse control. The persisted package state provides a runtime-changeable suspension control. The runtime must require both controls to permit execution:

```

environment allows internal
AND package is ACTIVE_INTERNAL
AND organization and role match package
AND definitions are ACTIVE and match manifest
AND no kill switch

```

This design permits immediate package suspension without deleting audit evidence or rolling back additive migrations.

11. Security Threat And Control Analysis

Threat	Current control	Required additional control
Direct activation without approval	DRAFT default and manifest check	Guarded release transition; remove direct --activate writes
Wrong code activated	Prompt/schema hashes	Commit SHA and complete signed manifest hash in activation package
Tenant scope widened	Environment allowlist	Immutable approved scope plus reconciliation and package check
Role scope widened	Empty-by-default role allowlist	Immutable role codes and approval-bound assignment
Expired approval remains active	None	Expiry/revocation check on transition and reconciliation
Owner unavailable	None	Accepted primary/backup assignment and coverage validation
Reconciler secret stolen	Constant-time comparison	Secret manager, rotation, network restriction, rate limiting, preferably OIDC
Scheduler silently stops	None	Durable heartbeat and missing-run alert
Alert transport silently fails	Durable pending row only	Dispatcher, retry, dead letter, secondary route, delivery SLO
Prohibited business write	Read-only registry and static scans	Retain; add DB mutation assertions to enabled E2E
Direct Prisma use by an agent	Prohibited patterns and narrow adapter	Retain; static import boundary and production credential separation
Cross-tenant replay or feedback	Scoped keys and foreign keys	Retain; enabled browser negative certification
Unsafe details in alerts	Safe summaries	Delivery redaction contract and canary tests
Evidence deletion during rollback	Additive migrations and retained runs	Runbook forbids deletion; retention check before retirement
CI certification reused for another build	None	Commit/hash-bound certification with expiry

For stronger production separation, use a release-control database credential that may update definition and activation tables while the normal application runtime can only read definitions and write run evidence. The agent itself never receives either credential.

12. Practical Implementation Plan

Step	Objective and changes	Owner	Verification and evidence	Fail/rollback rule
1. Repository and deployment inspection	Freeze the candidate commit; generate a complete machine-readable manifest; inventory current environments and deployment provider	Release preparer	Manifest artifact, commit SHA, current DB status report	Stop if worktree or deployment identity is ambiguous
2. Gap confirmation	Confirm the five blocked gates and record unavailable production evidence	Release owner	This report plus approved issue/register	Keep rollout off
3. Approval-record implementation	Add activation package, approval, owner, and certification schema; migrate; add protected permissions and append-only audit	Platform and security engineers	Schema, migration safety, unit and PostgreSQL tests	Roll back application code; do not roll back additive evidence tables
4. Guarded transition service	Implement state machine and remove direct activation status writes from provisioning	Platform engineer	Negative transition matrix; static direct-write gate	Definitions remain DRAFT
5. Ownership and runbooks	Enter actual people, backups, routes, window, and duty acknowledgements	Release owner	Accepted owner rows and signed RACI/runbooks	Missing owner blocks REVIEWED -> APPROVED
6. Reconciliation implementation	Extend Workflow Assurance with AGENT_RUNTIME checks; combine drift and abandoned-run reconciliation	SRE and platform engineer	Unit, PostgreSQL, drift injection, heartbeat tests	Detection-only mode if automatic suspension misbehaves
7. Scheduler and secret deployment	Deploy a 5-minute production-like schedule; configure OIDC or secret-manager bearer; add rotation and late-run alert	SRE	Infrastructure artifact, secret reference, two successful heartbeats	Disable job action, preserve detection; activation remains blocked
8. Alert transport	Implement shared delivery worker; configure primary/secondary routes, dashboards, thresholds, and named ownership	SRE and security	Delivered/acknowledged test alerts; retry and dead-letter proof	No healthy route means no activation
9. Pilot environment	Create isolated pilot and denied tenants/users; provision DRAFT then shadow through transition service	Pilot and QA owners	Seed report and tenant-scope assertions	Destroy only E2E fixtures; preserve release evidence
10. Playwright certification	Add enabled desktop/mobile, denial, failure, evidence, accessibility, suspension, and rollback matrix	QA owner	Required CI job, report hash, certification row	Any required failure invalidates certification
11. Product and security sign-off	Approve exact commit, manifest, scope, window, risks, owners, and evidence	Product and security approvers	Two current approval rows and evidence hashes	Rejection or expiry blocks activation
12. Controlled activation	Preflight; shadow rehearsal; activate one package for one organization and approved roles	Rollout owner	Audit event, active package, exact definitions, successful synthetic run	Rollback owner suspends on any High/Critical anomaly
13. Post-activation review	Monitor first hour/day/week; review incidents, feedback, SLOs, and residual risk	Pilot and release owners	Review record and continue/adjust/retire decision	Suspend if acceptance or safety thresholds fail

13. Controlled Activation Runbook

1. Confirm the candidate commit and deployed build identity match the activation package.
2. Confirm the package is PILOT_CERTIFIED.
3. Confirm product and security approvals are current, unrevoked, and hash-matched.
4. Confirm all six owner responsibilities have accepted primary and backup people.

5. Confirm the activation window is open.
6. Confirm the pilot organization and role list match deployment configuration.
7. Confirm two consecutive reconciliation heartbeats.
8. Confirm primary and secondary alert transports are healthy.
9. Confirm there are no open Critical or unaccepted High incidents.
10. Confirm the latest enabled-pilot Playwright certificate matches the same commit and manifest.
11. Confirm the kill-switch procedure was rehearsed.
12. Run provisioning in inactive or shadow mode through the guarded service.
13. Run one synthetic shadow execution and inspect run, evidence, redaction, and audit rows.
14. Invoke the guarded `ACTIVE_INTERNAL` transition with fresh authentication, idempotency key, and expected package version.
15. Verify one real pilot user can generate a brief and one denied user cannot.
16. Observe dashboards and alert routes continuously during the initial activation window.
17. Record the outcome and owner acknowledgement.

No step may be skipped. A failed preflight returns the package to the prior safe state.

14. Rollback And Emergency-Disable Runbook

Immediate rollback

1. Rollback or security owner activates the environment kill switch when available.
2. Invoke the guarded package suspension operation.
3. Atomically set the package to `SUSPENDED` and exact definitions to `PAUSED`.
4. Verify new runs are denied for pilot and non-pilot users.
5. Leave existing AgentRun, evidence, feedback, incident, approval, and audit records unchanged.
6. Open or update the assurance incident and assign it to the named owner.
7. Notify product, security, pilot, support, and on-call owners.
8. Capture the deployed commit, manifest, environment state, last successful run, reconciliation result, and alert-delivery evidence.

Recovery

1. Diagnose and fix the cause in a new candidate commit when code changes are needed.
2. Generate a new manifest and package for any changed code or scope.
3. Repeat approvals when manifest, tenant, roles, risks, or window changed.
4. Repeat enabled-pilot certification.
5. Rehearse shadow mode.

6. Resume only through the guarded transition service.

Do not recover by editing rows manually, deleting incidents, deleting failed runs, or reusing a certificate from another build.

15. Evidence And Audit Checklist

Release identity

- ☐ Candidate commit SHA recorded.
- ☐ Complete manifest JSON generated.
- ☐ Manifest hash recorded.
- ☐ Agent, skill, tool, schema, prompt, and model-provider identities recorded.
- ☐ Phase 2A provider remains none.

Approval and ownership

- ☐ Product approval valid.
- ☐ Security approval valid.
- ☐ Residual risks explicitly accepted.
- ☐ Pilot organization and roles explicitly approved.
- ☐ Activation window approved.
- ☐ Rollout owner and backup accepted.
- ☐ Rollback owner and backup accepted.
- ☐ Support owner and backup accepted.
- ☐ Pilot owner and backup accepted.
- ☐ Security incident owner and backup accepted.
- ☐ On-call route and backup accepted.

Operations

- ☐ Scheduler provider and cadence recorded.
- ☐ Secret or workload identity reference recorded.
- ☐ Secret rotation tested.
- ☐ Reconciliation heartbeat healthy.
- ☐ Manifest-drift detection tested.
- ☐ Critical-drift suspension tested.
- ☐ Primary alert delivered and acknowledged.
- ☐ Secondary alert delivered and acknowledged.
- ☐ Alert retry and dead letter tested.
- ☐ Dashboards and SLOs reviewed.

Certification

- ☐ Enabled desktop pilot passed.

- ☐ Enabled mobile pilot passed.
- ☐ Non-pilot tenant denial passed.
- ☐ Denied-role and missing-permission cases passed.
- ☐ DRAFT and manifest-mismatch cases passed.
- ☐ Stale, partial, empty, timeout, retry, and replay states passed.
- ☐ Evidence navigation remained tenant-protected.
- ☐ Audit and run provenance assertions passed.
- ☐ No prohibited business-table mutation occurred.
- ☐ Suspension, kill switch, and rollback passed.
- ☐ Accessibility and layout checks passed.
- ☐ Certification commit and manifest match the package.

Activation

- ☐ Package is PILOT_CERTIFIED.
- ☐ No open Critical incident.
- ☐ No unaccepted High incident.
- ☐ Preflight completed immediately before activation.
- ☐ Guarded activation audit event exists.
- ☐ Post-activation allow and deny probes passed.
- ☐ First-hour owner review completed.

16. Final Go/No-Go Checklist

Question	Current answer
Is the deterministic read-only engineering boundary verified?	Yes
Are the exact product and security approvals recorded?	No
Are all operational owners and backups named and acknowledged?	No
Is control-plane activation protected from direct script/database bypass?	No
Is a monitored reconciliation schedule deployed?	No
Is its production secret or workload identity configured?	No
Does reconciliation detect manifest, scope, approval, and certification drift?	No
Is an external alert transport deployed and owned?	No
Does a pending alert have a working dispatcher and escalation path?	No
Has the real enabled pilot passed Playwright on the exact manifest?	No
Has rollback been exercised against an enabled pilot?	No
May <code>--apply --activate</code> be run now?	No
May rollout be changed to <code>internal</code> now?	No

17. Final Decision And Next Executable Action

Decision: BLOCKED

Passed gates:

- deterministic provider-free architecture;
- tenant and actor context;
- RBAC and module control;
- read-only tool registry;
- prohibited business-action boundary;
- code/database manifest validation logic;
- bounded timeout;
- idempotent scoped replay;
- evidence, redaction, and safe failure persistence;
- abandoned-run reconciler implementation;
- static gates and focused automated tests;
- disabled and kill-switch browser behavior.

Blocked gates:

1. Recorded product and security approval.
2. Named and acknowledged operational owners.
3. Deployed control-plane reconciliation schedule and secret or workload identity.
4. Working production alert delivery and accountable ownership.
5. Enabled-pilot Playwright certification for the exact release.

The next technically executable action is:

Implement the narrow release-control slice: activation package, approvals, owner assignments, pilot certification record, guarded state-transition service, and a static gate that removes direct definition activation from the provisioning script.

In parallel, the organization must supply the actual product approver, security approver, rollout owner, rollback owner, support owner, pilot owner, security incident owner, backup coverage, pilot organization, approved roles, and activation window. Those identities cannot be inferred from code and must not be fabricated.

After that slice is verified, deploy reconciliation and alert delivery, build the enabled-pilot Playwright job, record the two approvals, and rerun 017-aqstoqflow-enterprise-release-gate.

18. Skill Output Contract

Selected skills: 016-aqstoqflow-ai-copilot-guardrails, then 017-aqstoqflow-enterprise-release-gate.

Files changed:

- what-next/agents-runtime/STOQUIFY_INTERNAL_ACTIVATION_GATE_UNBLOCKING_ANALYSIS_AND_EXECUTION_PLAN_2026-07-23.md

Gates passed: engineering architecture, tenant, RBAC, module, read-only integrity, idempotency, bounded timeout, typed failure, evidence, redaction, local persistence, static policy gates, focused automated tests, and kill-switch behavior.

Gates blocked: approval governance, named ownership, governed activation transition, deployed reconciliation, production alert delivery, and enabled-pilot certification.

Verification result: **BLOCKED for internal activation; ready to implement the release-control remediation slice.**

Next recommended numbered skill: remain on 017-aqstoqflow-enterprise-release-gate until all activation evidence is present. Do not advance to model-backed Phase 2B.